

lab1 项目组成和执行流

lab1的项目组成如下:

```
— Makefile
├─ kern
│   ├── debug
│   │   ├── assert.h
│   │   ├── kdebug.c
│   │   ├── kdebug.h
│   │   ├── kmonitor.c
│   │   ├── kmonitor.h
│   │   ├── panic.c
│   │   └─ stab.h
│   ├── driver
│   │   ├── clock.c
│   │   ├── clock.h
│   │   ├── console.c
│   │   ├── console.h
│   │   ├── intr.c
│   │   ├── intr.h
│   │   ├── kbdreg.h
│   │   ├── picirq.c
│   │   └─ picirq.h
│   ├── init
│   │   ├── entry.S
│   │   └─ init.c
│   ├── libs
│   │   ├── readline.c
│   │   └─ stdio.c
│   ├── mm
│   │   ├── memlayout.h
│   │   ├── mmu.h
│   │   ├── pmm.c
│   │   └─ pmm.h
│   └─ trap
│       ├── trap.c
│       ├── trap.h
│       └─ trapentry.S
├─ libs
│   └─ defs.h
```

```
|   |— elf.h
|   |— error.h
|   |— printfmt.c
|   |— riscv.h
|   |— sbi.c
|   |— sbi.h
|   |— stdarg.h
|   |— stdio.h
|   |— string.c
|   |— string.h
|— tools
   |— function.mk
   |— kernel.ld
```

内核启动相关文件

`kern/init/entry.S` : OpenSBI启动之后将要跳转到的一段汇编代码。在这里进行内核栈的分配, 然后转入C语言编写的内核初始化函数。

`kern/init/init.c` : C语言编写的内核入口点。主要包含 `kern_init()` 函数, 从 `kern/init/entry.S` 跳转过来完成其他初始化工作。

编译、链接相关文件

`tools/kernel.ld` : ucore的链接脚本(link script), 告诉链接器如何将目标文件的section组合为可执行文件, 并指定内核加载地址为0x80200000。

`tools/function.mk` : 定义Makefile中使用的一些函数

`Makefile` : GNU make编译脚本

其他文件

项目中还包括基础库 (`libs`)、设备驱动 (`kern/drive`)、内存管理(`kern/mm`)、异常处理 (`kern/trap`) 等文件, 这些文件的相关内容将在后续实验中进行完善和具体讲解。

执行流

完整流程

最小可执行内核的完整启动流程为:

加电复位 → CPU从0x1000进入MROM → 跳转到0x80000000(OpenSBI) → OpenSBI初始化并加载内核到e



详细步骤

第一步是**硬件初始化和固件启动**。QEMU 模拟器启动后，会模拟加电复位过程。此时 PC 被硬件强制设置为固定的复位地址 0x1000，从这里开始执行一小段写死的固件代码

(MROM, Machine ROM)。MROM 的功能非常有限，主要是完成最基本的环境准备，并将控制权交给OpenSBI。OpenSBI 被加载到物理内存的 0x80000000 处。

第二步是**OpenSBI 初始化与内核加载**。CPU 跳转到 0x80000000 处继续运行。OpenSBI 运行在 RISC-V 的最高特权级 (M 模式)，负责初始化处理器的运行环境。完成这些初始化工作后，OpenSBI 才会准备开始加载并启动操作系统内核。OpenSBI 将编译生成的内核镜像文件加载到物理内存的 0x80200000 地址处。

第三步是**内核启动执行**。OpenSBI 完成相关工作后，跳转到 0x80200000 地址，开始执行 kern/init/entry.S。在 0x80200000 这个地址上存放的是 kern/init/entry.S 文件编译后的机器码，这是因为链接脚本将 entry.S 中的代码段放在内核镜像的最开始位置。entry.S 设置内核栈指针，为C语言函数调用分配栈空间，准备C语言运行环境，然后按照RISC-V的调用约定跳转到 kern_init() 函数。最后，kern_init() 调用 cprintf() 输出一行信息，表示内核启动成功。